

THE EFFICIENT-AI RESEARCH TRACK

Month 4 — Working Like a Researcher

Tokenization → the framework transition → real scale on a GPU → reading papers cold.

Days 91–120, self-contained — plus the research map of cheap AI.

THE MONTH'S SHAPE

Months 1–3 you built everything by hand — that was the point. Month 4 you graduate: proper tokenization, an autograd framework (earned, not assumed), your first GPU-scale run of nanoGPT — where your raw-vs-clean data hypothesis finally gets tested with real compute — and your first two papers read cold. The closing chapter maps the whole cheap-AI research landscape and ranks where YOU can attack it.

Prerequisites: Months 1–3, especially ai_13 (tiny GPT), ai_15 (data tools), ai_16 (distillation). Needs: free Colab/Kaggle GPU access for Week 15.

WEEK 13 — Tokenization: Beyond Characters

Goal: your models have eaten characters — simple, but wasteful. Build Byte-Pair Encoding (BPE) from scratch: the compression trick that decides what a model's "words" are.

13.1 Why characters were holding you back

Character models spend capacity relearning that t-h-e forms a unit, thousands of times. And every character costs a full forward pass to generate — "the" is 3 passes for one trivial word. Two wastes: **modeling waste** (capacity spent on spelling instead of meaning) and **compute waste** (passes per unit of text).

The idea: let the DATA decide the vocabulary. Count which adjacent pairs occur most, merge the top pair into a new token, repeat. Frequent chunks ("the", "ing", "tion") become single tokens; rare words stay decomposable into pieces. That's **BPE** — a compression algorithm moonlighting as a vocabulary designer.

```
Hand-trace on the text: "low low low lower lowest"
start (chars):  l o w _ l o w _ l o w _ l o w e r _ l o w e s t
count pairs:    (l,o) x5   (o,w) x5   (w,_) x3   (w,e) x2 ...
MERGE 1: (l,o) -> "lo"    text: lo w _ l o w _ l o w e r _ l o w e s t
count again:    (lo,w) x5   ...
MERGE 2: (lo,w) -> "low"   text: low _ l o w _ l o w e r _ l o w e s t
MERGE 3: (low,_) -> "low_" ... and "low" is now ONE token. The data chose it.
```

```
# ai_18_bpe.py -- the whole learner in ~15 lines
def learn_bpe(text, num_merges):
    tokens = list(text)                # start as characters
    merges = []
    for _ in range(num_merges):
        counts = {}
        for a, b in zip(tokens, tokens[1:]): # count adjacent pairs
            counts[(a, b)] = counts.get((a, b), 0) + 1 # your Stage-0 dict!
        if not counts: break
        best = max(counts, key=counts.get) # most frequent pair
        merges.append(best)
        merged, i = [], 0                 # rewrite text with the merge
        while i < len(tokens):
            if i+1 < len(tokens) and (tokens[i], tokens[i+1]) == best:
                merged.append(tokens[i] + tokens[i+1]); i += 2
            else:
                merged.append(tokens[i]); i += 1
        tokens = merged
    return merges, tokens
```

Recognize the machinery? Pair-counting is your Stage-0 counting dict; the greedy merge is a loop. You could have written this in week one — what's new is knowing WHY it matters.

WHY / FAILURE / MEASURE

Why: shorter sequences (2–4x fewer tokens for English) = fewer passes AND attention's n^2 shrinks quadratically — tokenize better, pay 4–16x less attention compute for the same text. **Failure:** vocab too big = rare tokens starve (your Week-5 finding); too small = back to spelling. Numbers/code tokenize weirdly — inspect them. **Measure:** compression ratio (chars per token) AND downstream validation loss at equal COMPUTE, not equal steps.

Week 13 exercises

1. Train 500 merges on your corpus. Print the first 30 and last 30 merges. What kind of chunks come early vs late?
2. Retrain your tiny GPT on BPE tokens (vocab ~500) vs characters, SAME multiply budget (fewer steps for BPE — sequences shrink, but vocab grows the final matrix: account for both with ai_9). Which wins at equal compute?
3. Tokenize "2026-07-03" and "torch.nn.Linear" with your English-trained BPE. Ugly? That ugliness is a real production issue — note why domain-matched tokenizers exist.

ANSWERS

1: early = spaces+common bigrams ("e ", "th"); late = whole words/suffixes ("the ", "ing"). **2:** usually BPE wins clearly at equal compute — that's the point. **3:** dates/code fragment into awkward pieces; mismatched tokenizers quietly tax quality forever.

WEEK 14 — The Framework Transition (Earned)

Goal: adopt PyTorch — and because you built backprop by hand, autograd will feel like a colleague automating your bookkeeping, not magic.

14.1 What autograd actually does

Everything you did manually in ai_6 — recording what flowed forward, applying the two blame rules right-to-left — PyTorch does automatically. Every tensor operation is recorded on a tape; `loss.backward()` replays the tape backward applying your blame rules; gradients appear in `.grad`. That's the whole mystery. You've hand-executed the algorithm it automates, which puts you in the rare position of knowing exactly what it's doing and when to distrust it.

```
import torch
w = torch.tensor([1.0], requires_grad=True) # "track blame for this"
x, target = torch.tensor([2.0]), torch.tensor([12.0])
guess = w * x * 2.0 # the 2-layer chain from Month 1!
loss = (guess - target) ** 2
loss.backward() # blame flows backward
print(w.grad) # -64. -- verify by YOUR hand rules
```

TRY IT 1 (THE RITE OF PASSAGE)

Verify that -64 by hand with your Month-1 blame rules (chain: $\text{guess} = w \cdot x \cdot 2$, so $d \text{ loss} / d w = 2 \cdot \text{error} \cdot x \cdot 2$). If your paper number matches the machine's, you own autograd.

14.2 Rebuild the tiny GPT in PyTorch — and prove it's the same

The week's project: `ai_19_torch_gpt.py` — your ai_13 architecture, expressed in `nn.Module`, `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`. The critical step is the **parity test**, a real research-engineering skill:

1. Copy your NumPy weights into the torch model (transpose carefully — `nn.Linear` stores weights as (out, in): your rows-as-neurons convention survives, one transpose at the border).
2. Same input → both models. Outputs must match to ~6 decimals.
3. One training step each, same batch, same step size → weights must still match.

When the parity test passes, you have PROVEN your three months of hand-built understanding and the industrial tool are the same machine. Frame that moment. Then enjoy the payoff: training that took minutes now takes seconds (fused C++/GPU kernels under the same math), and architecture changes become one-liners.

FAILURE MODES TO EXPECT

Shape mismatches from the (out,in) convention; forgetting `optimizer.zero_grad()` (blame ACCUMULATES by default — a silent wrong-answer machine); train/eval mode differences later. Log each bug you hit: your future debugging speed is built from this list.

Week 14 exercises

1. Time NumPy ai_13 vs torch ai_19 (CPU) on identical training: steps/second. Log the speedup.
2. Break autograd on purpose: insert a `.detach()` mid-network. What happens to learning upstream of it, and why is this ALSO a feature? (Hint: your frozen distillation teacher.)
3. Reproduce your Month-2 ablation table (Week 7 ex.1) in torch in one evening — the same experiment that took days by hand. That ratio is why frameworks exist.

ANSWERS

- 1:** typically 5–20x on CPU. **2:** upstream weights stop learning — detach cuts the blame tape; it's exactly how you freeze a teacher (ai_16 did this implicitly). **3:** the point is the felt ratio; expect same conclusions, hours less pain.

WEEK 15 — Real Scale: nanoGPT and Your Data Hypothesis

Goal: a free GPU, a real 10M-parameter model, and the experiment your roadmap promised in Month 1: raw vs deduplicated data, at a scale where the answer means something.

15.1 The GPU, demystified

A GPU is a matrix-multiply machine: thousands of simple cores doing your `W @ x` rows in parallel. Free access: Google Colab or Kaggle notebooks (T4-class GPU, hours per day — enough for everything this week). The entire code change: `model.to("cuda")` and move your batches there too. Expect another 10-50x over CPU torch. Your Stage-0 vectorization instinct, `ai_9` multiply-counting, and batch training were all preparation for this machine.

15.2 nanoGPT: reading someone else's codebase

nanoGPT (Karpathy's minimal GPT training repo) is the standard small-scale research vehicle. This week you don't write the model — you READ one, a core researcher skill. Checklist: find where attention lives (recognize your `ai_12`, plus tricks: fused QKV, dropout), where the training loop lives (your `ai_14` plus mixed precision and schedules), where data enters. For every trick you don't recognize, one log line: what it is, what it's for. Expect ~5 unknowns; that's the reading list working.

15.3 THE experiment: raw vs deduplicated, at scale

Your Week-9 result was suggestive but tiny. Now the honest version:

1. Corpus: a few hundred MB of public text (e.g. a Project Gutenberg dump or similar open corpus).
2. Arms: (a) raw, (b) deduplicated with your `ai_15` tools scaled up (near-dup detection matters at this size), (c) raw randomly cut to (b)'s size — the control you already know is non-negotiable.
3. Model: nanoGPT small config (~10M params), identical config and token budget for all arms, ONE run each (GPU-hours are your budget now — say so in the write-up).
4. Report: validation loss on a clean held-out slice + your verbatim-recall memorization metric from Week 9 + samples.

WHY THIS IS REAL RESEARCH

Dedup-at-scale results in the literature were run at sizes you can't touch — but the small-scale regime is genuinely under-mapped, and "does the effect hold/strengthen/vanish at 10M params on this corpus type?" is a question with no published answer for your exact setup. Prediction first. Whatever you find, the write-up (question, method, control, result, limits) is your first piece of research at meaningful scale — portfolio centerpiece.

Week 15 exercises

1. Before training, predict all three validation losses to 2 decimals, sealed in the log. Score your calibration after.
2. Measure tokens/second on GPU vs your CPU torch model. Combine with `ai_9` math: how many multiplies/second is the GPU doing? What fraction of its advertised peak is that? (Welcome to utilization — a whole efficiency field.)
3. Overfit-one-batch (your Week-8 move) on nanoGPT before any long run. It catches config bugs that would waste your GPU budget.

ANSWERS

1: the habit is the answer — calibration improves only when scored. **2:** typically 10-30% of peak for naive setups; the gap is why kernels/fusion/precision research exists. **3:** if it can't hit ~0 on one batch, fix before spending GPU-hours.

WEEK 16 — Reading Papers Cold

Goal: the last skill of the apprenticeship. Two classic papers, read with a method — and you now personally own every concept inside them.

16.1 The three-pass method

Pass 1 (10 min): title, abstract, all figures, conclusion. Write the claimed contribution in ONE sentence of your own words before continuing. **Pass 2 (1 hr):** full read, skipping proofs; log every term you can't define (each becomes a lookup). **Pass 3 (only for papers you'll build on):** reproduce the core table/figure at your scale, or trace the method by hand with tiny numbers — your standard move since Month 1.

16.2 Paper 1: "Training Compute-Optimal LLMs" (Chinchilla, 2022)

Why this one first: it's your Week-8 scaling capstone at planetary scale, and its punchline is pure efficiency: the field had been training models too BIG on too LITTLE data — for a fixed compute budget, a smaller model on more tokens wins (rule of thumb born here: ~ 20 tokens per parameter). Reading assignment: find the figure showing equal-compute curves; then check YOUR Month-2 four-dot plot and Week-8 ex.2 (params vs data) against its logic — were your tiny models over- or under-fed? Write the verdict in the log.

16.3 Paper 2: "Distilling the Knowledge in a Neural Network" (Hinton et al., 2015)

The original distillation paper — and you have ALREADY reproduced its core idea (ai_16). Reading assignment: find where it introduces temperature-softened targets (your $T=2$ line!) and the mixing weight (your α). Pass-3 exercise: their famous demo removed a class from training and let soft labels teach it anyway — design the character-level analogue for your models (drop a character from the student's hard labels; can the teacher's soft spreads still teach it?) and run it. That's reproduction WITH a twist: the researcher's basic unit of work.

16.4 The habit going forward

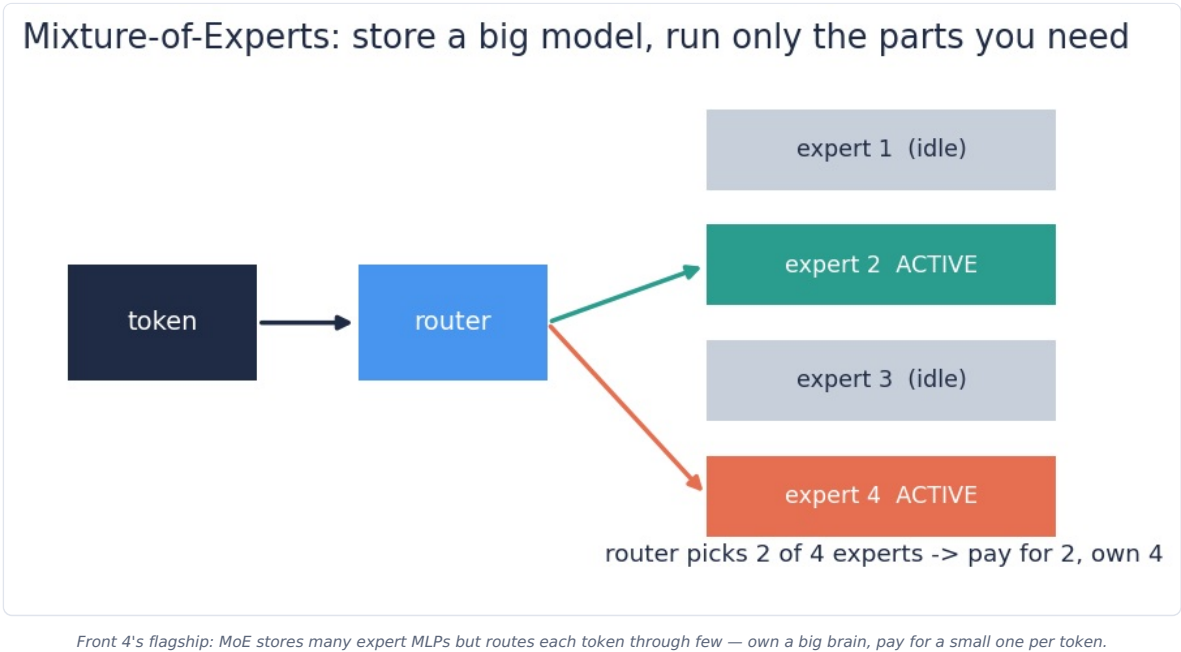
One paper per two weeks, permanently, chosen from the map in the next chapter. Log format per paper: one-sentence claim · method in 3 bullets · what I'd test at my scale · one doubt. Doubt is mandatory — papers are arguments, not scripture.

THE MAP — Where Cheap-AI Research Lives (and Where YOU Can Attack)

The whole landscape of "same quality, less cost," organized by what it attacks — each area rated for solo-researcher tractability with your compute (one GPU, patience, honest controls).

The seven fronts

Front	The bet	Named techniques you'll meet	Solo tractability
1. Data	better food beats more food	dedup, quality filtering, curriculum ordering, synthetic data, data mixing ratios	★★★ highest — cheap to test, under-mapped at small scale, your tools exist (ai_15)
2. Knowledge transfer	big models can compress themselves into small ones	distillation (yours!), on-policy/sequence-level distillation, self-distillation, teacher ensembles	★★★ — works at your scale; open questions everywhere (what makes a good teacher?)
3. Numeric precision	models don't need 32-bit numbers	8/4-bit inference (yours!), quantization-aware training, outlier handling, low-bit TRAINING (frontier)	★★ — inference-side very tractable; training-side needs care but small-scale findings transfer surprisingly well
4. Sparsity	most of a model is idle most of the time	pruning (yours!), structured sparsity, Mixture-of-Experts	★★ — pruning science easy; real MoE speedups need engineering, but small MoE behavior is studyable
5. Architecture	attention's n^2 is not a law of nature	linear attention, sliding windows, state-space models (Mamba), hybrids	★ — ideas testable small, but the field's hard lesson is that small-scale wins often vanish at scale; enter with humility
6. Training dynamics	reach the same loss in fewer steps	optimizers, LR schedules, initialization, hyperparameter transfer (muP), early stopping (yours!)	★★ — experiments cheap; noise is the enemy — multiple seeds or don't publish it to yourself
7. Inference craft	generation itself is wasteful	speculative decoding (small model drafts, big model verifies), KV-cache compression, batching/caching	★★★ — speculative decoding is a beautiful solo project: you HAVE a big and small model pair already



Your positioning (honest strategic advice)

- **Lead with fronts 1, 2, 7.** They're compute-light, under-explored at small scale, and compound with each other (curate → train small → distill → speculative-decode). They are also exactly what your capstone practiced.
- **The niche that fits you:** "how much quality can a 10–100M-param DOMAIN model reach under an explicit budget, using stacked efficiency techniques with honest controls?" Almost nobody publishes carefully at that scale; hobbyists don't control, labs don't bother. Careful small-scale work with sealed predictions and size-matched controls is a real, credible contribution lane.
- **The trap to avoid:** architecture novelty (front 5) is the moth-flame of new researchers — seductive, and where small-scale results most often mislead. Visit later, with scars.
- **The compounding question** behind your whole track, worth keeping on the wall: efficiency techniques are usually studied one at a time — *how do they interact?* Does distillation make quantization easier (soft-trained students may be smoother)? Does dedup change what pruning removes? Your v1.0 codebase can run these 2x2 interaction studies TODAY, and interaction effects are chronically under-published.

Reading list by front (your next 6 months of paper habit)

Front	Start with
Data	the deduplication paper ("Deduplicating Training Data Makes LMs Better"), then a data-mixture paper
Transfer	Hinton distillation (done, W16), then a sequence-level distillation paper
Precision	the QLoRA paper (quantization + LoRA in one — two of your tools meet)
Sparsity	the Lottery Ticket Hypothesis paper (pruning's most provocative claim — testable at YOUR scale)
Architecture	the Mamba paper — read for the PROBLEM statement (attention's costs) even before the solution
Dynamics	Chinchilla (done, W16), then a learning-rate-schedule study
Inference	the speculative decoding paper — then build it: your capstone student drafts, your Month-2 teacher verifies

Month 4 closing checklist

- ☐ Week 13: my BPE learner is built, and my GPT wins at equal compute on its own tokens
- ☐ Week 14: parity test passed — my NumPy GPT and my torch GPT are provably the same machine
- ☐ Week 15: raw-vs-dedup ran at 10M params with a size-matched control, sealed predictions, and a write-up
- ☐ Week 16: two papers read with the three-pass method; the missing-character distillation twist ran
- ☐ Map: I chose my first front (1, 2, or 7) and my next experiment has a hypothesis written down

Codebase now: ai_18_bpe.py → ai_19_torch_gpt.py → nanogpt-experiment/ + writeups, tagged v1.1.

Month 5+ horizon: your chosen front, two-week paper cadence, interaction studies (the 2x2s), speculative decoding build, and the first experiment designed entirely by you — question, method, control, prediction — with me as skeptic rather than guide. That's the handover the whole track was for.